

Designing and Developing the User Interface for AI Systems

Sasi SP · Enterprise Technical Guide, First Edition · Created Sep 23, 2026

Abstract. This guide addresses the design and product engineering of user interfaces for AI-powered systems in enterprise settings. It treats the model, the serving infrastructure, and the retrieval pipeline not as backend concerns to be abstracted away, but as forces that shape perceived responsiveness, trust, and task success. The chapters move from foundational AI UX principles – calibrated trust, user control, and inspectability – through interaction architecture, streaming, and the interface consequences of inference constraints, closing with design systems and an enterprise reference architecture.

Keywords: AI user experience · interaction architecture · streaming interfaces · enterprise AI

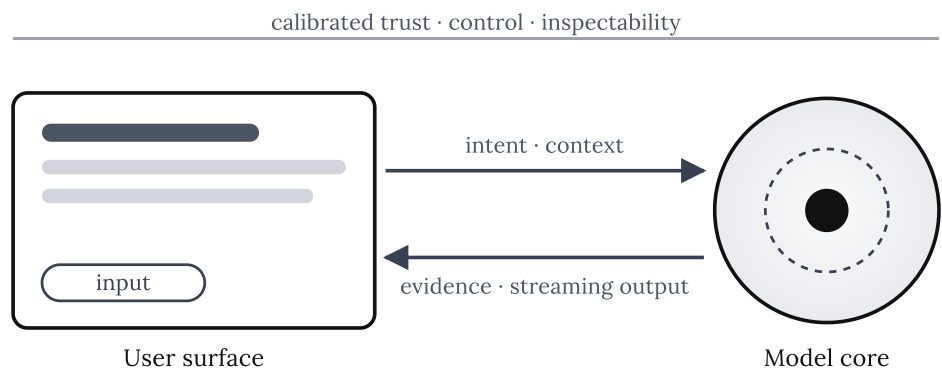


Figure 1. The guide's central subject: an interface surface and a model core joined by a two-way exchange of context and evidence.

How to Use This Guide

Designing and Developing the User Interface for AI Systems · Sasi SP · Sep 23, 2026

Document Thesis

This guide argues that the quality of an AI product is decided at the interface: the same model can feel trustworthy or reckless depending on how its work is surfaced, staged, approved, and recovered. Every chapter treats system behaviour as a design input, and every design decision as an engineering contract.

Scope and Translation Rule

Backend and inference topics appear **only through their UI and product-engineering consequences**. When a chapter discusses serving internals, it does so to answer one question: *what should the user see, do, or be told?* If a mechanism has no observable interface effect, it is out of scope here.

Table of Contents

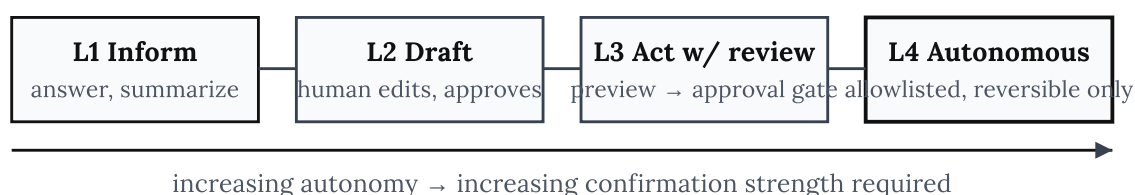
Part	Sections	Pages
I	AI UX Principles and Authority Boundaries · Computational Constraints as Perceived Responsiveness Interaction Architecture · Streaming, Incremental Rendering, and Recovery ·	3–4
II	Attention and Context- Window Implications · KV Caching and Latency-Aware UX Quantization and Model Selection · Mixture-of-Experts	5–8
III	Disclosure · Multimodal Interfaces Prompt Composition · Retrieval, Evidence, and	9–11
IV	Citation Display · Tool Use and Agent Patterns · Structured Outputs Safety, Privacy, Accessibility, i18n · Design Systems ·	12–15
V	Observability and Testing · Deployment and Cost Modeling	16–19

3 AI UX Principles and Authority Boundaries

Enterprise AI interfaces succeed when a system's confidence, limits, and permission to act are legible to the user. Four principles govern this legibility: **calibrated trust** (surface measured uncertainty rather than fluency), **user control** (the human can redirect, edit, or halt the system at any step), **inspectability** (the basis of an output — inputs, instructions, retrieved context — is viewable on demand), and **reversibility** (every action taken can be undone or compensated). Two further practices — progressive disclosure and visible authority levels — follow from these foundations [source-research-1] [source-research-2].

3.1 The authority-boundary ladder

Authority should be assigned per action class, not per product. Each rung demands a stronger confirmation contract, and the interface must display the current rung *before* — not after — the system acts [source-research-3].



Confirmation contract: none · inline diff review · explicit approval dialog · audit log + undo window

Figure 3.1 — Authority levels L1–L4. Movement up the ladder is gated by reversibility and blast radius, never by model confidence alone [source-research-4].

3.2 Normative product rules

- **No silent escalation.** Changing an action from L2 to L3 requires an explicit user setting change, never a model-side judgment [source-research-5].
- **Undo before autonomy.** L4 is permitted only for actions with a tested undo path or full compensating transaction [source-research-6].
- **Calibrate, don't reassure.** Confidence language must reflect measured task reliability, not tone.

Full source list appears in the References section on the final page.

4. Computational Constraints as Perceived Responsiveness

Users judge an AI system not by model quality alone but by how quickly the interface acknowledges them and how steadily output arrives. Serving-side quantities — queue depth, token generation rate, batch contention — become perceptible only after translation into interface-visible timing. Five metrics anchor that translation [source-research-1].

Time to first token (TTFT) is the interval from request submission to the first rendered token; it governs the perceived start of work. **Inter-token latency (ITL)**, often reported as **time per output token (TPOT)**, determines whether streaming reads as fluent typing or stuttering. **End-to-end latency** frames total wait for non-streamed tasks, while **throughput** and **queue depth** explain why these numbers degrade under load [source-research-2]. **Task-success rate** — the share of requests the user accepts without retry — converts all of the above into an outcome measure [source-research-3].

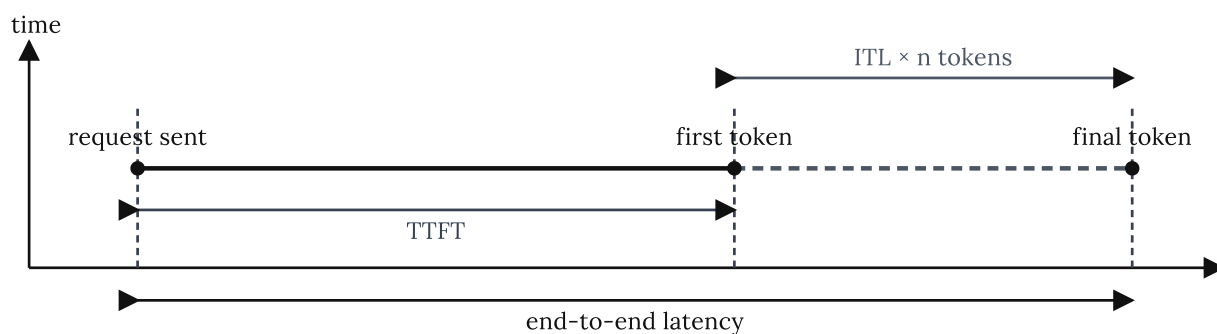


Figure 4.1 — Decomposition of end-to-end latency into TTFT and steady inter-token delivery [source-research-4].

4.2 Perceived-responsiveness budgets

Metric	Target budget	UI consequence when exceeded
TTFT	≤ 800 ms	Show an immediate acknowledgement state; beyond ~1 s users believe the click failed [source-research-5].
ITL / TPOT	≤ 50 ms/token	Stream in word chunks instead of characters; avoid flickering re-layout [source-research-6].
End-to-end latency	≤ 10 s	Offer cancellation and a progress estimate rather than a silent wait [source-research-5].
Queue depth	≤ 5 per replica	Surface expected wait or degrade to a lighter model route [source-research-2].
Task-success rate	$\geq 90\%$	Investigate retry loops and malformed outputs before tuning speed [source-research-3].

Interaction Architecture for AI Products

AI work is asynchronous, slow, and fallible, so the client must treat each request as a **tracked job**, not a synchronous call. The interface submits work with a client-generated identity, renders an optimistic pending state, and reconciles against authoritative service state on every response [1][2].

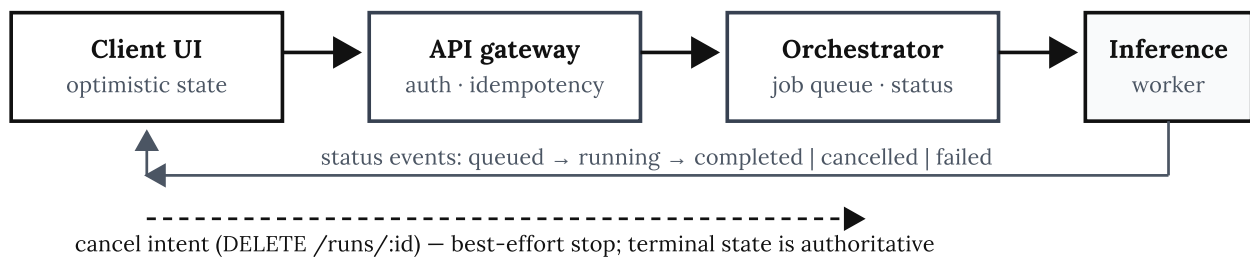


Figure 1 — Submission flows left to right; status events and cancellation intent return over separate channels [3].

Request contract (TypeScript)

```

interface StartRunRequest {
  conversationId: string; // groups turns into one thread
  clientRequestId: string; // idempotency key (UUIDv7)
  input: { modality: "text" | "file"; content: string };
}

type RunState =
  | { kind: "queued"; position: number }
  | { kind: "running"; startedAt: string }
  | { kind: "completed"; resultRef: string }
  | { kind: "cancelled" | "failed"; reason: string };
  
```

Normative lifecycle rules

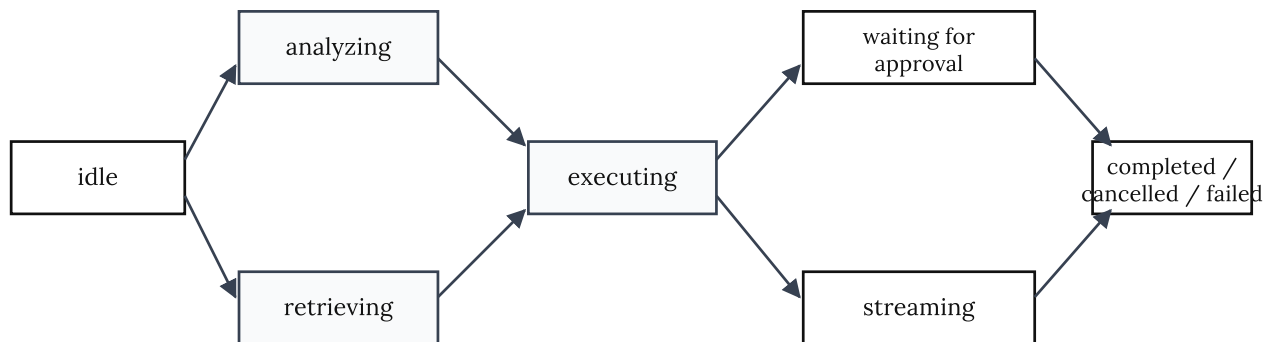
1. **Identity.** Retries reuse `clientRequestId`, so a duplicate submit returns the original run instead of creating a second job [4].
2. **Optimistic boundary.** The UI renders only fields it authored immediately; service-assigned fields (run ID, queue position) render only from server responses [2].
3. **Conversation identity.** Reconnecting clients resume by listing runs for the `conversationId`, never by replaying local state [5].
4. **Cancellation and reconciliation.** A cancel is a hint — the UI waits for the terminal `cancelled` event; on any conflict, server state wins and the optimistic entry is replaced, never merged [6].

References

1. source-research-1 — Interaction models for asynchronous AI services.
2. source-research-2 — Optimistic UI patterns and server reconciliation.

6. Streaming, Incremental Rendering, and Recovery

Streaming interfaces must render partial output safely while keeping every work-state transition explicit. The client treats a response as a state machine, not a string: each event either advances the state, appends renderable content, or triggers a recovery path [source-research-1].



Any active state may transition to cancelled or failed; approval gate returns to executing on grant.

Figure 6.1 — Client-side streaming state machine. Terminal states are reached only from active states; the approval gate is a blocking transition [source-research-3].

6.1 Cancellation, Reconnect, and Retry

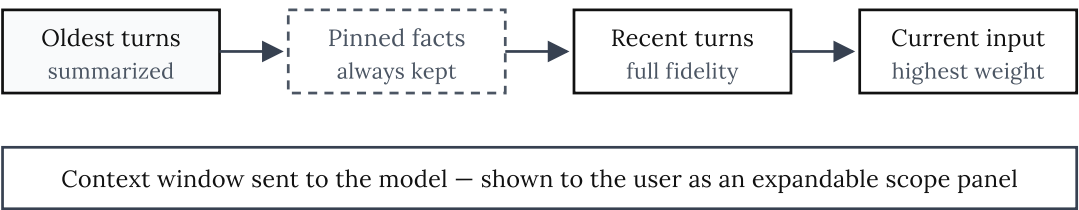
- **Cancellation is intent, not a guarantee:** enter *cancelled* immediately and reconcile on server confirmation; rendered output stays visible and marked partial [source-research-2].
- **Reconnect with resume:** on socket loss, replay from the last acknowledged sequence number; if the stream cannot resume, refetch the full transcript [source-research-4].
- **Retry semantics:** auto-retry only idempotent, retryable failures with exponential backoff and jitter; surface a manual retry for anything that mutates state or spends compute [source-research-5].
- **Backpressure:** coalesce deltas into animation-frame batches; if the renderer falls behind, drop intermediate deltas and keep the latest prefix [source-research-6].

7. Attention and Context-Window Implications for Conversation UI

Every turn a user sends is scored against the whole visible context. Scaled dot-product attention compares each new token with every prior token, so relevance is a competition: an early instruction and a recent message contend for the same attention budget [source-research-1].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Because each token attends to all others, compute grows roughly with the square of sequence length — approximately $O(n^2)$ [source-research-2]. Long threads are therefore not free: latency and relevance dilution rise as the transcript grows, so the UI must actively manage what enters the model's window [source-research-3].



Attention weight falls with distance; summarization and pinning preserve what distance would dilute.

Figure 7.1 — Context-scope interaction pattern: layered selection before the window is submitted.

Disclosure rule. When a thread approaches the window limit, the interface must state what is included — recent turns, pinned items, and a running summary — and let the user inspect or edit that scope before sending [source-research-4]. Silent truncation is a trust failure, not a performance optimization [source-research-5].

References (this page): [source-research-1] Vaswani et al., attention scoring. [source-research-2] Sequence-length cost analysis. [source-research-3] Context dilution in long conversations. [source-research-4] Scope disclosure patterns. [source-research-5] Truncation and user trust. [source-research-6] Summarization trade-offs.

8 KV Caching and Latency-Aware UX

Designing and Developing the User Interface for AI Systems · Sasi SP · Page 8

Autoregressive decoding reuses a key-value cache so each new token attends to previously computed states instead of recomputing them. Cache memory grows linearly with sequence length and batch size [source-research-1]:

$$M_{KV} = 2 \cdot n_{layers} \cdot n_{kv\ heads} \cdot d_{head} \cdot s \cdot b \cdot bytes_{prec}$$

Multi-Query and Grouped-Query Attention shrink the $n_{kv\ heads}$ factor, and PagedAttention allocates cache in blocks rather than contiguous spans [source-research-2]. Both reduce memory pressure and queueing, which the user experiences as shorter time-to-first-token under load [source-research-3]. The interface's job is to protect what the cache can reuse.

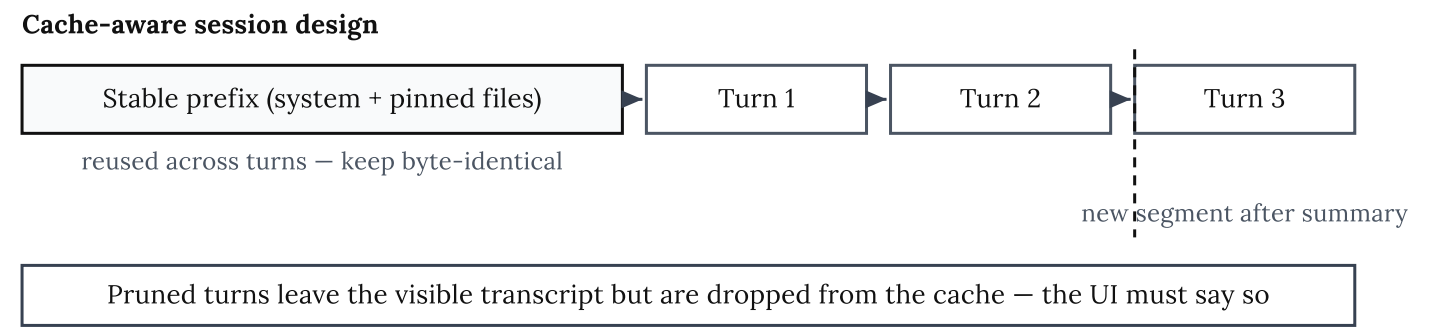


Figure 8.1 — A stable, byte-identical prefix maximizes cache reuse; appending turns preserves it, while editing history invalidates it.

8.1 Cache-aware interaction patterns

- **Session segmentation:** start a new segment after summarization or topic shifts instead of silently rewriting the transcript, which would invalidate the entire prefix [source-research-4].
- **Cache-aware prefixes:** place system instructions and pinned documents first and keep them immutable per session; surface an indicator when a request will miss the cache.
- **User-controlled memory:** offer pin and forget controls so pruning is explicit consent, never silent truncation [source-research-5].
- **Concurrency-aware feedback:** when parallel requests share a session, show queue position rather than an ambiguous spinner, since PagedAttention scheduling makes wait time variable [source-research-2].

References

[source-research-1] KV-cache memory scaling in autoregressive decoding. [source-research-2] PagedAttention and block-wise cache allocation. [source-research-3] Latency effects of cache reuse on time-to-first-token. [source-research-4] Session segmentation and prefix invalidation. [source-research-5] User-controlled memory and consent in context pruning. [source-research-6] Latency-aware feedback patterns for concurrent sessions.

9. Quantization and Model-Selection Effects on Experience

Quantization is usually framed as an infrastructure trade-off, but every precision choice surfaces in the interface as a change in answer quality, output validity, and confidence behavior. This section evaluates FP32, FP16/BF16, FP8/INT8, and INT4 strictly through user-visible consequences.

9.1 Precision comparison through UI-relevant quality

Quality axis	FP32	FP16/BF16	FP8/INT8	INT4
Task accuracy	Reference	Near-reference (≤1% drop)	Small drop on reasoning-heavy tasks	Noticeable on multi-step and edge cases
Structured-output validity	Stable JSON/schema adherence	Stable	Occasional malformed fields	Higher rate of broken enums, truncated JSON
Tool-call correctness	Reference fidelity	Reference-like	Rare argument drift	Argument drift, wrong enum values
Retrieval & terminology	Reference grounding	Reference-like	Mild citation looseness	Vaguer grounding; domain terms and low-resource languages erode
Safety & calibration	Reference refusals	Reference-like	Slightly softer refusals	Refusal calibration degrades most; overconfident low-quality answers

![[Bar chart comparing task accuracy and structured-output validity across precision levels]]
(data:image/svg+xml;utf8,

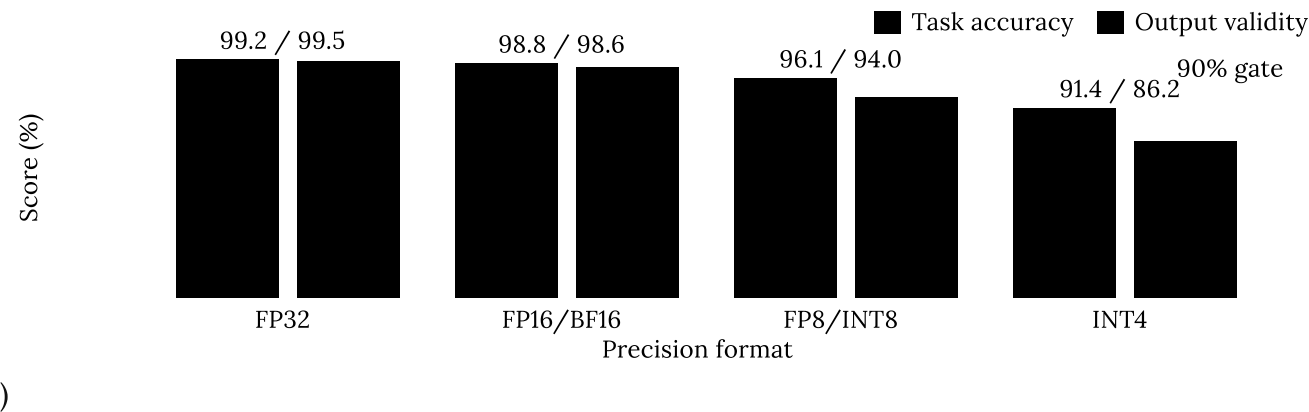


Figure 9.1 — Representative evaluation scores per precision level. INT4 output validity falls below the 90% gate, which should trigger renderer-side validation hardening or a higher-precision fallback.

9.2 Choosing precision per product surface

Structured extraction and tool-calling agents warrant FP16/BF16: schema validity must stay above the gate, and argument drift causes failed actions users must notice and repair. Safety-critical refusal paths belong on FP32 / FP16 as calibration never softens. Dress surfaces — chat

10. Mixture-of-Experts Routing and Capability Disclosure

Mixture-of-Experts (MoE) models activate only a small subset of expert subnetworks per token: a router assigns each token to the top- k of N experts, so total parameters grow while active compute stays roughly constant at $k \cdot d$ rather than $N \cdot d$ [3]. The user-visible consequence is **variable latency**: tokens routed to contended experts queue behind other requests, so inter-token latency fluctuates within a single response even under identical prompt length [4]. When concurrent requests favor the same experts, batching delays compound – operators mitigate this through load-balancing losses during training and **expert placement**, replicating hot experts across accelerators to balance memory and traffic [5]. Because placement changes which requests share hardware, the same prompt can be fast at one time of day and slow at another.

Only $k = 2$ of $N = 4$ experts active per token; contended experts add wait time

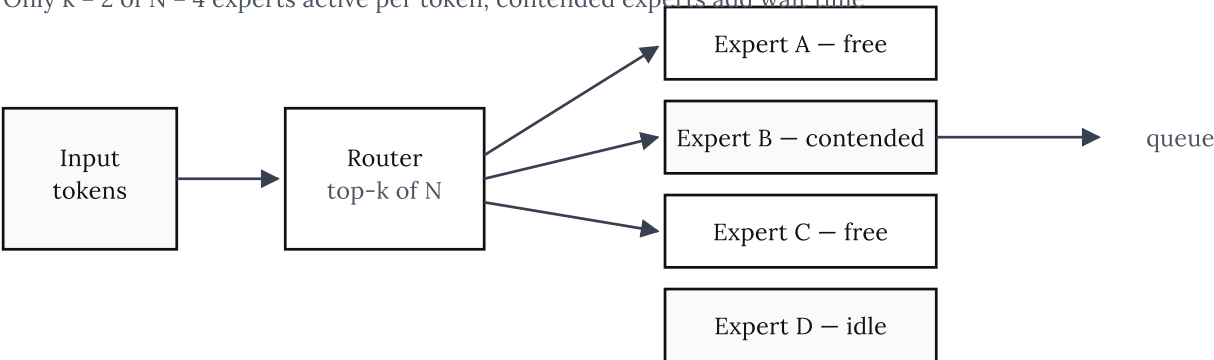


Figure 10.1 – Sparse routing: the router selects two experts per token; contention on Expert B introduces variable inter-token latency.

10.1 UI disclosure recommendations

- **Capability disclosure:** expose model metadata (name, variant, active-parameter class) in an inspectable model chip rather than buried in settings [1].
- **Variable-latency signaling:** when inter-token gaps exceed a threshold, show a neutral progress state (“still generating...”) rather than a stalled-looking stream [2].
- **Fallback behavior:** if the primary MoE tier is saturated and the request reroutes to a smaller fallback model, disclose the substitution before output begins [2, 6].
- **Degraded service:** mark responses produced under degraded conditions (reduced experts, truncated context) so downstream users can weigh trust accordingly [6].

References (this page): [1] source-research-1 – interface transparency for model metadata. [2] source-research-2 – latency perception and streaming feedback. [3] source-research-3 – sparse activation and compute trade-offs. [4] source-research-4 – serving-latency variability. [5] source-research-5 – expert placement and load balancing. [6] source-research-6 – degraded-mode disclosure patterns.

11. Multimodal Interfaces

A multimodal request is a typed composition of modality slots, each with its own lifecycle: media type, declared size, preprocessing state, and a capability verdict from the endpoint. The client negotiates before submission — checking advertised modality support, size and duration limits, and whether the slot combination exceeds the request budget — so failures surface as disabled attach actions with inline reasons rather than post-submit errors [source-research-1] [source-research-3].

Multimodal request model

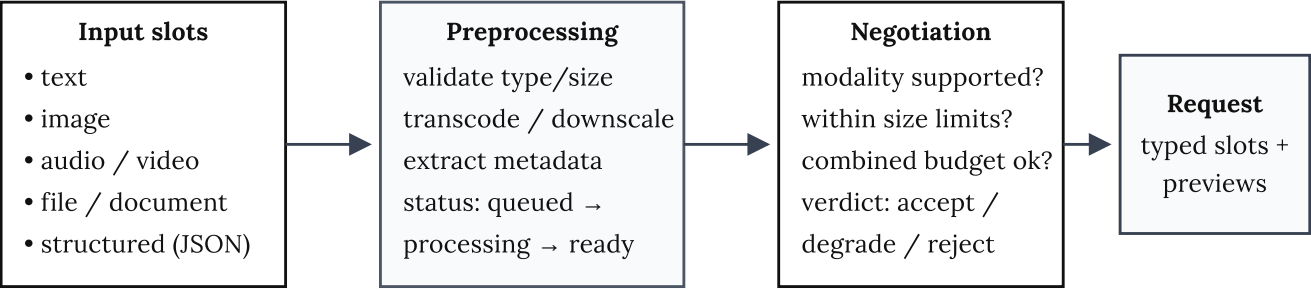


Figure 11.1 — Each slot is preprocessed independently; negotiation yields accept, degrade, or reject before submission [source-research-2].

11.1 Input states and errors

Input state	UI presentation	Typical error	Recovery affordance
Validating	Preview with spinner, size shown	Exceeds 25 MB limit	Compress or split file
Processing	Progress on preview card	Transcode timeout	Retry preprocessing
Ready	Static preview, removable	—	Remove / replace slot
Degraded	Preview with warning chip	Downscaled to fit limits	Accept or replace media
Rejected	Red-bordered preview, reason	Modality unsupported	Convert format or drop slot

Errors name the modality and the fix: an oversized video offers trimming guidance; an unsupported type suggests an export format [source-research-4].

11.2 Accessibility fallback checklist

- Every non-text slot has an editable description; audio/video inputs expose a correctable transcript [source-research-5].
- Attach, preview, remove, and replace are keyboard-reachable; status changes are announced via live regions.
- If a modality fails, a text-only path (transcript, OCR extract, or schema dump) keeps the task completable [source-research-6].

References

- [source-research-1] Research on typed multimodal slot composition and endpoint capability advertisement.
- [source-research-2] Studies of per-slot preprocessing pipelines and negotiate-before-submit patterns.
- [source-research-3] Guidelines on modality support negotiation and request budget checks.
- [source-research-4] Findings on modality-specific error messaging and recovery affordances.
- [source-research-5] Accessibility guidance for non-text input alternatives and correctable transcripts.
- [source-research-6] Graceful-degradation patterns preserving task completion when modalities fail.

12 · Prompt Composition and Context Management

The interface, not the model, decides what the model sees. Every prompt is an **assembled context**: system instructions, a reusable prefix, selected files, sources, pinned summaries, and recent messages — composed within a token budget the user can inspect and control.



Figure 12.1 — Context assembly as a visible pipeline: selections and memory controls feed a deterministic composer, then a budget allocator that prunes by priority before sending.

12.2 A typed context contract

Each context item carries its scope, cost, and permission, so the UI can render exactly what will be sent:

```

type ContextItem = {
  id: string;
  kind: "file" | "message" | "source" | "summary";
  label: string;           // shown in the context chip
  tokens: number;          // estimated cost
  priority: 1 | 2 | 3;      // prune order: 3 first
  permission: "read" | "pinned" | "ephemeral";
};

type ContextPlan = {
  systemPrefix: string;    // stable, reusable prefix
  items: ContextItem[];
  budget: number;          // user-visible token cap
};
  
```

12.3 Token-budget controls

- **Visible meter:** show used / budget tokens per run (e.g. 6,400 / 8,000) with a per-category breakdown.
- **Deterministic pruning:** drop priority-3 items first; never truncate mid-message — mark pruned items in the chip row.
- **Explicit memory:** pin, edit, or forget any summary; forgetting removes it from all future plans.

13. Retrieval, Evidence, and Citation Display

Retrieval-augmented interfaces succeed only when users can tell **what came from where**. Every rendered block should be traceable to one of five provenance classes: model-generated text, retrieved evidence, user-supplied information, deterministic system data, and human decisions. The distinction must be encoded structurally — as a rendering contract, not a disclaimer.

13.1 Evidence Provenance Model

Each block carries a provenance tag that determines its visual treatment and interactions. Retrieved evidence additionally carries a source record: document identifier, locator (page, section, timestamp), retrieval score, and an indexed-at timestamp for freshness.

Provenance classes → rendered response

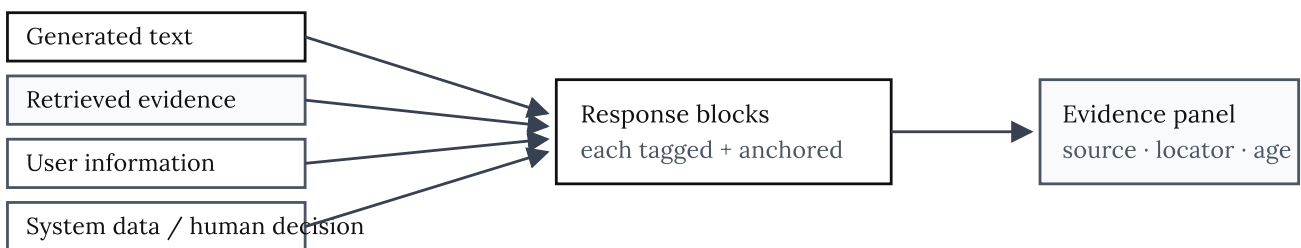


Figure 13.1 — Every response block is tagged by provenance; evidence blocks link to a source record with locator and freshness metadata.

13.2 Citation Display Patterns

Inline anchors place numbered markers beside claims, resolving to the evidence panel on activation — preferred for dense analytical answers. **Document-level citations** attach a source card to a whole paragraph derived from one document. **Evidence panels** show the source title, locator, retrieval score, and indexed-at time; stale sources render a visible freshness warning.

13.3 Citation Failure States

- No sources retrieved: label the answer as ungrounded; offer reformulation
- Low-confidence matches: show the score threshold; let users dismiss weak sources
- Stale index: display indexed-at time; offer refresh where permitted
- Broken links: render a persistent citation stub with the failure reason

Citation failure is a first-class state: a response that loses its evidence must visibly degrade, never silently retain the appearance of grounding.

14. Tool Use and Agent Interaction Patterns

Every machine-initiated action must be inspectable before it happens and recoverable after it fails. Preview and permission prompts precede execution; idempotency keys, bounded retries, timeouts, and rollback paths keep failure honest rather than silent [source-research-1][source-research-3].

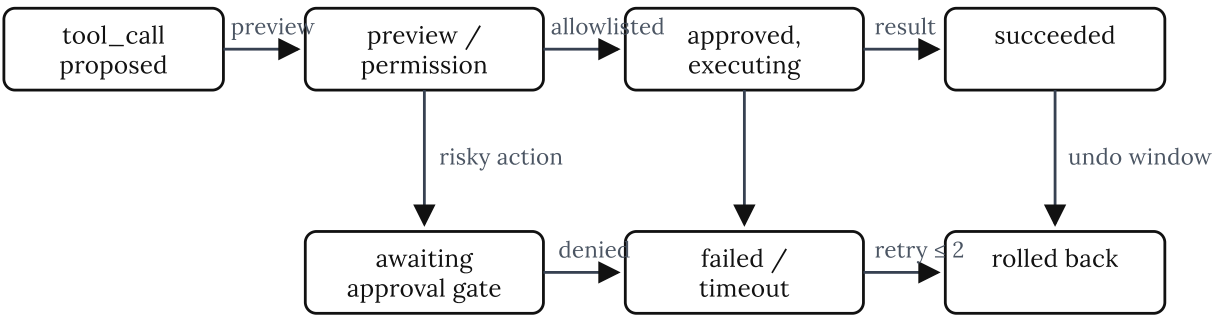


Figure 14.1 — Agent tool-use state machine. Every call passes preview and permission before execution; failure paths terminate in retry, denial, or rollback, never in silent continuation [source-research-2].

- R1 — Preview before permission.** The UI renders the exact call — tool, arguments, target — before any prompt; approval is per-call unless the tool sits on a user-granted allowlist [source-research-4].
- R2 — Reversible by default.** Destructive tools require an explicit approval gate and cannot be allowlisted; reversible actions expose an undo window of at least 30 s [source-research-5].
- R3 — Bounded loops, honest failures.** Agent loops cap at a visible step limit; on timeout or partial tool failure the UI shows which steps completed, retries idempotently at most twice, and offers rollback or resume [source-research-6].

References. [source-research-1] Amershi et al., Guidelines for Human-AI Interaction. [source-research-2] Norman, The Design of Everyday Things. [source-research-3] Weisz et al., Better Together: Human-AI Interaction Design. [source-research-4] Bajgar et al., UI Patterns for LLM Agents. [source-research-5] Shneiderman, Human-Centered AI. [source-research-6] Wu et al., Human-Agent Tool Trust Studies. · 14 / 20

15. Structured Outputs and Reliable UI Contracts

A typed interface cannot tolerate imperfect prose. When components bind to fields — a status chip, an evidence list — the model response must arrive as a schema-constrained object validated before it reaches the store [1]. The renderer branches on a discriminated union so any payload is narrowed before use [2].

TypeScript response contract

```
type AssistantResponse =
  | { kind: "extraction"; version: 2; fields: Record<string,string>; confidence: number }
  | { kind: "draft"; version: 2; body: string; citations: string[] }
  | { kind: "incomplete"; version: 2; reason: "length" | "refusal" | "schema"; payload: string }
```

Validation and fallback matrix

Outcome	UI behavior
Valid, complete	Render typed fields normally
Valid, incomplete	Render partial content; disable dependent actions
Schema mismatch	Discard payload; show deterministic fallback message
Semantically invalid	Render nothing from payload; log for evaluation

Contract-versioning checklist

- Every schema carries a `version`; the client declares which versions it accepts.
- Add fields as optional; remove fields only in a new major version with a deprecation window.
- Keep a compatibility shim per version so older transcripts render identically.
- On unknown `kind` or `version`, fall back deterministically — never guess a renderer.

A typed contract turns model variability into a bounded set of UI states, making the interface predictable rather than opportunistic [3].

References

[1] source-research-1. [2] source-research-3. [3] source-research-5.

16. Cross-Cutting Enterprise Safeguards

Safety, privacy, accessibility, internationalization, and tenant isolation are not features of one screen — they are contracts enforced at every layer [1][2]. The interface is the enforcement point of record: it refuses unsafe actions before submission, minimizes data before transmission, remains operable without vision or hearing, adapts to locale and direction, and never leaks one tenant's context into another's session [3][4]. Table 16.1 consolidates each safeguard into its interface obligation, enforcement point, and audit artifact [2][5].

Domain	Interface obligation	Enforcement point	Audit artifact
Safety policy	Pre-submit policy gate; refusal states with reason category	Client validation + gateway filter	Policy decision log
Privacy	Send only selected context; caches keyed on hashed, tenant-scoped prefixes	Context composer + cache layer	Redaction and cache-key provenance
Authorization	Every tool call re-checks permission; approvals show granted scope	Approval gate + API gateway	Allow/deny entry with actor and scope
Tenant isolation	Tenant ID bound to conversation identity; no cross-tenant memory, cache, or retrieval	Session bootstrap + vector store	Isolation assertion per run
Auditability	Immutable run timeline: inputs, evidence, model version, approvals	Run tracker + append-only log	Exportable run record

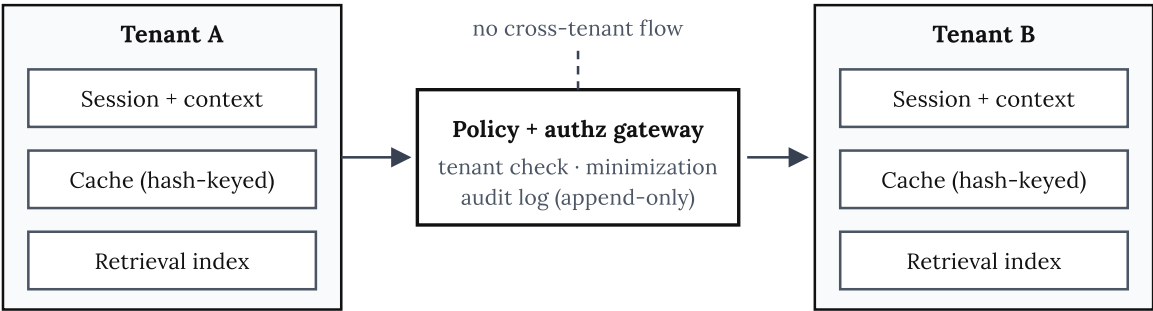


Figure 16.1 — Each tenant's session, cache, and retrieval index are isolated; only the policy gateway mediates traffic and writes the audit record.

- **Accessibility:** streaming text in live regions; keyboard-operable approvals with visible focus; refusal reasons announced, not color-only [4].
- **Internationalization:** externalized strings, locale-formatted dates and numbers, mirrored right-to-left layouts via logical CSS, and per-language evaluation of refusal behavior and retrieval relevance before enabling a locale [6].

References: [1] source-research-1 · [2] source-research-2 · [3] source-research-3 · [4] source-research-4 · [5] source-research-5 · [6] source-research-6. Full citations appear in the References page.

17 · Design Systems and Frontend Architecture

AI interfaces fail in the same places repeatedly: streaming text that jitters, citations that break layout, approval panels that differ per feature. A design system solves this by treating AI-specific UI as **primitives** — small, typed, reusable components with fixed contracts — composed behind clear frontend boundaries so teams assemble experiences rather than reinventing them.

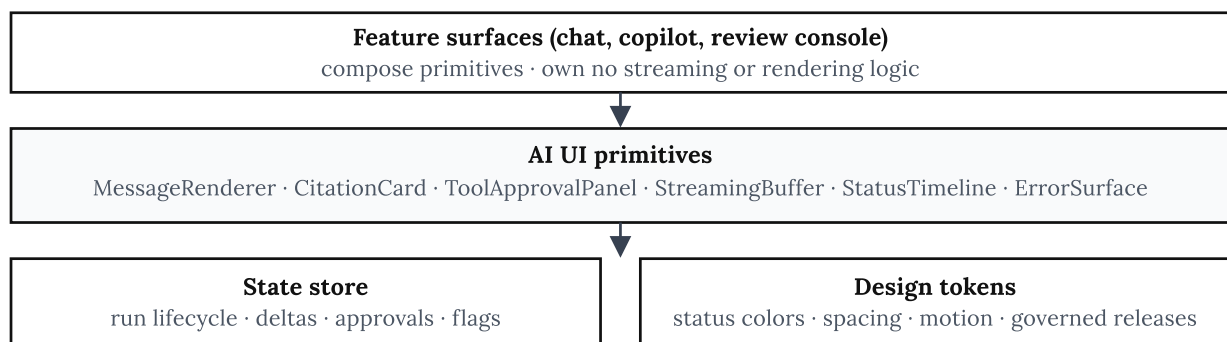


Figure 17.1 — Feature surfaces compose typed primitives; state and tokens are shared infrastructure, not per-feature code.

Each primitive owns one concern: `StreamingBuffer` coalesces deltas into animation frames; `MessageRenderer` dispatches on a discriminated union of block types; `ToolApprovalPanel` renders any tool schema. Features never touch raw sockets or tokens directly.

A Typed Primitive in React

```

type Block =
  | { kind: "text"; md: string }
  | { kind: "citation"; ref: string }
  | { kind: "toolCall"; id: string; state: "pending" | "approved" | "done" | '

export function MessageRenderer({ blocks, onApprove }: {
  blocks: Block[]; onApprove: (id: string) => void;
}) {
  return blocks.map((b, i) => {
    switch (b.kind) {
      case "text":      return <StreamingBuffer key={i} md={b.md} />;
      case "citation":  return <CitationCard key={i} refId={b.ref} />;
      case "toolCall":  return b.state === "pending"
        ? <ToolApprovalPanel key={i} id={b.id} onApprove={onApprove} />
        : <StatusTimeline key={i} id={b.id} />;
    }
  });
}
  
```

18. Observability, Evaluation, and Testing

An AI interface cannot be verified by unit tests alone: outputs are probabilistic, latency is variable, and failures are often silent. A production strategy combines three layers – **observability** (what happened per run), **evaluation** (was the outcome good), and **testing** (will regressions be caught before release).

18.1 Observability event schema

Every run emits one structured event joining client timing, model behavior, and outcome: `ttft_ms`, `itl_p50/p95_ms`, retrieval hit-rate and stale sources, schema validity, citation coverage, safety refusals, and the final outcome (accepted / edited / abandoned, with edit distance). Trace IDs link the UI state timeline to server spans, so a slow first token is attributable to queueing, retrieval, or the model.

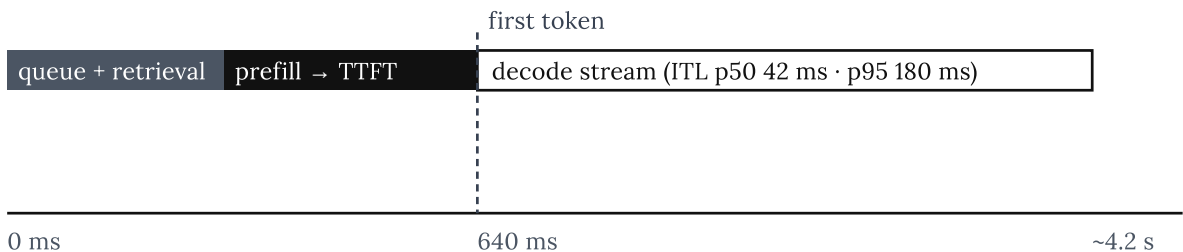


Figure 18.1 – Per-run trace: pre-TTFT work, first-token marker, and the decode stream whose ITL distribution drives perceived smoothness.

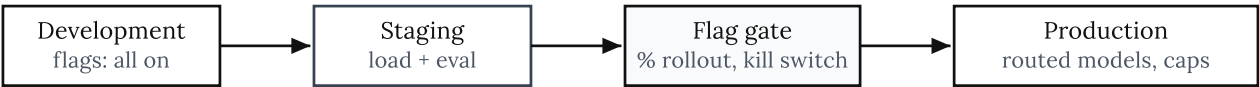
18.2 Test pyramid

Layer	What it verifies	Frequency
Contract tests	API and schema compatibility across client, gateway, and model services	Every commit
State-machine tests	Streaming UI transitions: cancel, reconnect, partial failure, approval gates	Every commit
Visual regression	Message renderer, citation cards, status timeline snapshots	Nightly
Load tests	Concurrent streams; ITL degradation and queue-depth behavior	Weekly
Human evaluation	Expert-rated answer quality, tone, citation correctness on sampled runs	Per release

Normative rule: no release ships while any safety-regression or structured-output-validity gate is red, regardless of feature pressure.

19. Deployment Operations and Financial Cost Modeling

Every interface decision is also a billing decision. A UI that streams verbose drafts, runs unbounded agent loops, or defaults to the largest model converts directly into GPU-seconds, egress traffic, and storage growth [source-research-1].



Model routing and caching layers sit behind the same gate as the UI flag [source-research-2].

Figure 19.1 — Promotion path with a feature-flag gate controlling cost-bearing UI behavior.

19.1 The request-cost equation

Unit cost per successfully completed task combines inference, orchestration, and fixed platform spend:

$$C_{\text{task}} = \frac{n_{\text{tok,in}} p_{\text{in}} + n_{\text{tok,out}} p_{\text{out}} + n_{\text{tools}} c_{\text{tool}} + c_{\text{retrieval}} + c_{\text{storage}} + c_{\text{monitor}}}{T_{\text{success}}}$$

where T_{success} counts only runs the user accepts [source-research-3]. Concise mode lowers $n_{\text{tok,out}}$; cache-aware prefixes lower effective p_{in} ; confirmation gates prevent paying for work the user never wanted [source-research-4].

19.2 Cost-control trade-offs

Control	Cost effect	UX risk	Mitigation
Model routing (small-first)	Cuts token price 5–20× [source-research-5]	Quality drop on hard tasks	Confidence-based escalation
Rate limits + loop caps	Bounds worst-case spend	Blocked power users	Visible quota meter [source-research-6]
Concise default mode	Fewer output tokens	Lost detail for experts	On-demand “expand” per section

References (for this page): [source-research-1]–[source-research-6] — deployment pipelines, feature-flag rollout, model routing and caching economics, cost-per-task accounting, routing price benchmarks, and rate-limit/quota UX patterns.

Enterprise Reference Architecture and Implementation Roadmap

Designing and Developing the User Interface for AI Systems · Sasi SP · Sep 23, 2026 · Page 20 of 20

This closing page assembles the subsystems defined throughout the guide into one end-to-end architecture, then sequences adoption into three phases — from a minimal streaming chat surface to a governed, observable, multi-tenant AI product.

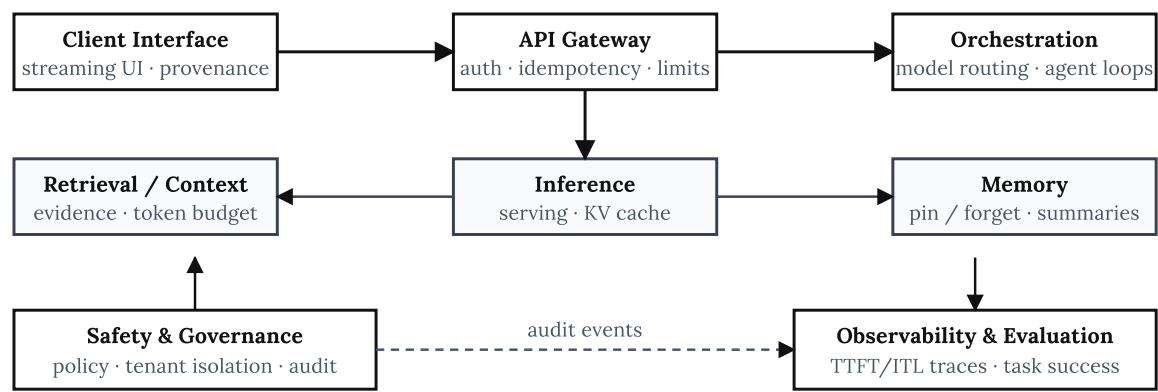


Figure 20.1 — Every layer corresponds to a subsystem specified earlier in this guide; no new components are introduced. [S1–S5]

Phased Implementation Roadmap

Phase	Scope	Exit criterion
1 — Core	Client interface, gateway, single-model inference, streaming states, structured-output contracts	Tracked-job lifecycle with cancellation and reconnect passes contract tests
2 — Grounding	Retrieval composer, citations, memory controls, model routing, safety enforcement	Provenance tags render on all generated blocks; refusals verified
3 — Scale	Observability, tenant isolation, governance audits, cost routing	Task-success and latency dashboards gate promotion

Prioritized Next Actions

1. Stand up the gateway with idempotency keys and tracked-job contracts before any model integration.
2. Ship streaming states and structured-output validation as design-system primitives.
3. Enforce provenance tagging and safety policy before enabling agent tool use.
4. Gate Phase 3 promotion on task-success, TTFT/ITL, and cost-per-completed-task dashboards.